ML-projekt: Förutsägelse av trafikträngsel

Nov 10, 2026 · @Arezoo

Sammanfattning

Vi förutsäger trafikträngsel (fyra klasser) från trafiksensorer i nordiska städer. Slutmodellen, en Random Forest, förutsäger trängseln **en timme framåt** med accuracy 0,81 ± 0,02, jämfört med 0,63 för gissningen att trängseln blir samma som nu.

Vägen dit gick via ett viktigt fynd: de första modellerna nådde 99,99 % eftersom fyra mätkolumner i praktiken avslöjade svaret (dataläckage). När de togs bort sjönk accuracy till en realistisk nivå, och modellen använder nu tid, väder och vägtyp.

Därefter bytte gruppen target till trängseln en timme framåt (`future_congestion_level`). Det är ett mer användbart problem, där dagens mätningar får användas. Datan blir mindre (cirka 2 200 rader), men trädmodellerna slår ändå baselinen tydligt. Slutsatserna finns i avsnitt 14.

1. Dataset och problemformulering

Vi valde NordCauc Urban Mobility Intelligence (Kaggle), filen `traffic_flow_sensors.csv`, med 200 000 mätningar från 160 sensorer under 2023–2025. Problemet är att förutsäga trängselnivån `congestion_level`: free_flow, light, moderate eller heavy.

Motivering till datasetet

- Det passar vår utbildningsinriktning mot mobilitetstjänster.
- Det är stort nog för att träna och testa pålitligt.
- Det innehåller både tid, plats, väder, vägtyp och trafikmätningar, vilket ger flera möjliga problem att formulera.

Motivering till `congestion_level` som target

- Det är en tydlig och praktiskt användbar fråga: hur mycket trängsel är det?
- Fyra namngivna klasser gör det till ett **klassificeringsproblem**, inte regression. Därför använder vi klassificerare och mäter med accuracy och classification report, inte MAE eller R^2.
- Datasetet har också kolumner som `ai_predicted_congestion`. De togs bort eftersom de är en annan modells svar och skulle göra uppgiften meningslös.

2. Datarensning och förbehandling

Efter rensningen återstår 184 025 rader utan tomma värden, där alla kolumner är siffror.

Steg	Vad vi gjorde	Varför
Datakvalitet	Behöll bara rader med `data_quality_flag == "good"` (184 025 av 200 000). Bort föll `sensor_fault` (9 928) och `missing_partial` (6 047).	Felaktiga eller ofullständiga mätningar lär modellen fel mönster.
Borttagna kolumner	`sensor_id`, `country`, `district`, `timestamp`, `date`, `latitude`, `longitude`, `ev_count_15min`, `ai_predicted_volume`, `ai_predicted_congestion`, `ai_model_version`, `data_quality_flag`	Id-nummer och koordinater säger inget generellt om trängsel. `ai_`-kolumnerna är en annan modells svar (läckage). `data_quality_flag` är konstant efter filtreringen.
Tomma värden	Kontroll med `isnull().sum()`: 0 i alla kolumner	Inga rader behövde fyllas i eller tas bort.
Text i X	`pd.get_dummies(X)` på `city`, `day_of_week`, `season`, `road_type`, `weather_condition`, `incident_type`	Modeller kräver siffror. One-hot ger varje kategori en egen kolumn, utan påhittad ordning.
Text i y	`map` (free_flow=0, light=1, moderate=2, heavy=3)	Tydliga etiketter. Klassificerare klarar även text, så steget är valfritt.
Uppdelning	`train_test_split`, 80/20, `stratify=y`, `random_state=42`	Testdata som modellen aldrig sett. `stratify` ger samma klassfördelning i båda delarna.

Ordningen spelar roll: rensning, sedan X/y, sedan encoding, sedan uppdelning och sist träning. Notebooken ska köras med **Restart Kernel and Run All Cells** före varje push, så att alla resultat motsvarar samma körning.

3. Baseline och första modell

Baseline-modellen gissar alltid den vanligaste klassen och får accuracy 0,462. Varje riktig modell måste slå det värdet för att ha lärt sig något.

```
base_line = DummyClassifier(strategy="most_frequent").fit(x_train, y_train)
accuracy_score(y_test, base_line.predict(x_test))   # 0.462
```

Motivering: `DummyClassifier` tittar inte på X alls, bara på y. Den visar alltså vad man får "gratis". Vi använder inte `DummyRegressor`, MAE eller R^2, eftersom problemet är klassificering.

Den första riktiga modellen var en `RandomForestClassifier` med alla kolumner. Den fick accuracy **0,9999** och nästan 1,00 i precision och recall för alla fyra klasser. Ett så perfekt resultat är orimligt för verklig trafikdata och fick oss att leta efter fel (avsnitt 4).

4. Dataläckage: varför fyra kolumner togs bort

Fyra kolumner mäts samtidigt som trängseln och avslöjar i praktiken svaret. När vi tog bort dem sjönk accuracy från 0,9999 till 0,862, vilket visar att modellen tidigare läste av en regel i stället för att lära sig mönster.

Vad är dataläckage? Modellen får information som den inte skulle ha i en verklig situation, eller som nästan är facit. Resultatet ser perfekt ut men säger ingenting om hur modellen fungerar på riktigt.

Hur vi upptäckte det

1. Accuracy 0,9999 var orimligt högt för verklig trafikdata.
2. Caroline påpekade att hög beläggning (`occupancy_rate`) och låg hastighet (`avg_speed_kmh`) i praktiken *är* definitionen av heavy traffic.
3. Simon testade ett Decision Tree med **bara** `occupancy_rate` och fick 99,9 %. En enda kolumn räckte alltså för att nästan alltid gissa rätt.
4. Vi tog bort kolumnerna stegvis och mätte effekten:

Borttagna kolumner	Accuracy (Random Forest)
Inga	0,9999
`occupancy_rate`, `avg_speed_kmh`	0,986
+ `vehicle_count_15min`, `truck_count_15min`	0,862

Motivering för varje kolumn

- `occupancy_rate`: andel av vägen som är upptagen. Hög beläggning *är* trängsel. Ensam ger den 99,9 %.
- `avg_speed_kmh`: låg hastighet *är* trängsel. Trängselnivån är troligen beräknad från hastighet i förhållande till hastighetsgräns.

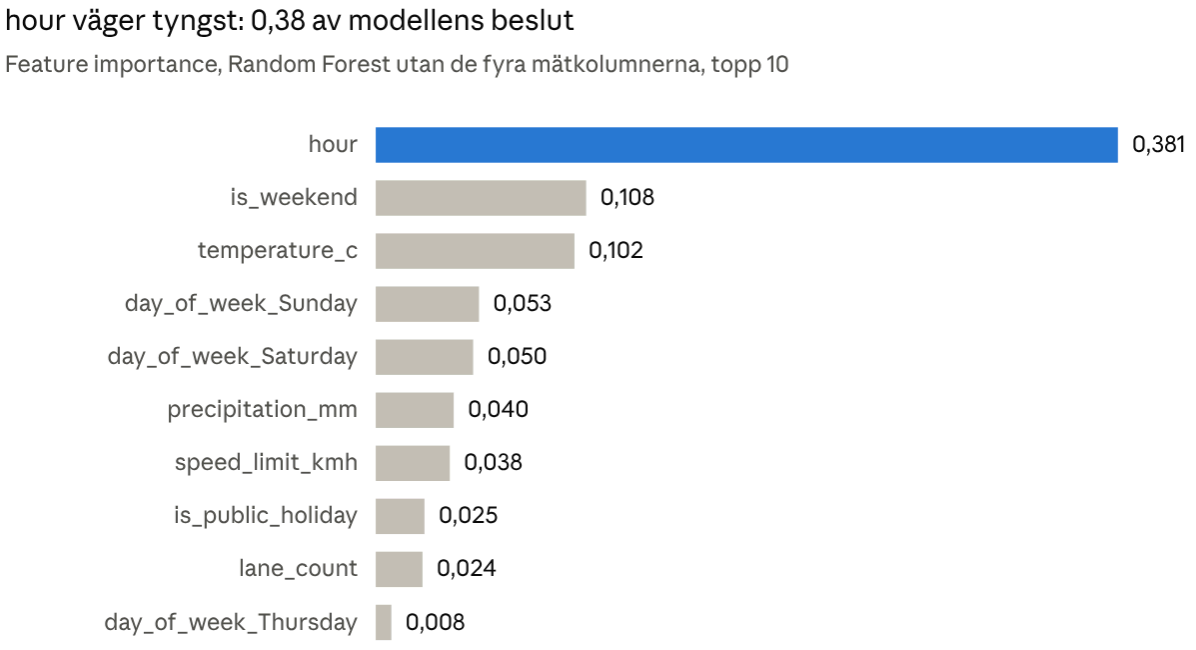
- `vehicle_count_15min`: antal fordon som räknas under samma 15 minuter som trängseln. Efter att de två första togs bort låg accuracy kvar på 0,986, alltså läckte det fortfarande.
- `truck_count_15min`: samma typ av samtidig mätning som `vehicle_count_15min`.

Den gemensamma principen: alla fyra mäts *samtidigt* som det vi vill förutsäga. Kvar blir bara det man vet *i förväg*: tid (`hour`, `day_of_week`, `is_weekend`, `is_public_holiday`, `season`), plats (`city`), väg (`road_type`, `speed_limit_kmh`, `lane_count`), väder (`temperature_c`, `precipitation_mm`, `weather_condition`) och `incident_type`.

Frågan modellen besvarar blir då: *kan vi förutsäga trängsel utifrån tid, plats, väg och väder, utan att mäta trafiken?*

5. Feature importance: läckan är borta

Ingen kolumn dominerar orimligt längre. Tiden på dygnet är viktigast, och ungefär 60 % av vikten ligger på tidsrelaterade kolumner (hour, helg och veckodag).



Från notebookens feature_importances_ · Random Forest, 10 viktigaste kolumnerna

Motivering till varför resultatet är rimligt: trafik följer tid på dygnet (rusningstid och natt) och helg eller vardag. Det förklarar också varför free_flow fick recall 1,00, eftersom natten nästan alltid är fritt flöde.

Om temperaturen: den kan väga tungt av två skäl. Temperatur hänger ihop med tid på dygnet och årstid, så modellen kan använda den som en indirekt klocka. Dessutom ger Random Forest ofta högre vikt åt kolumner med många olika värden. `permutation_importance` är en mer pålitlig kontroll om gruppen vill verifiera det.

6. Jämförelse av modeller

Decision Tree gav bäst resultat (0,90), före Random Forest (0,86). Båda slår baseline med god marginal, medan Logistic Regression hamnar klart efter.

Modell	Accuracy	Hur den fungerar
Decision Tree	0,900	En serie ja/nej-frågor, som ett flödesschema
Random Forest	0,862	100 träd som röstar, majoriteten vinner
Logistic Regression	0,647	Vikt per kolumn, ger en sannolikhet per klass
Baseline (DummyClassifier)	0,462	Gissar alltid vanligaste klassen

Alla modeller tränades och testades på samma data (X utan de fyra mätkolumnerna, 80/20-uppdelning, `random_state=42`).

Motivering till modellvalen: vi jämför en enkel trädmodell, en ensemble av träd och en linjär modell. Då ser vi om problemet kräver icke-linjära samband.

Tolkning

- **Varför slår Decision Tree Random Forest?** Datan är troligen syntetisk och skapad med regler (t.ex. viss tid och vardag ger heavy). Ett enda djupt träd kan hitta exakt de reglerna. Dessutom väljer Random Forest vid varje fråga bland ett slumpmässigt urval av kolumner. Efter `get_dummies` finns många kolumner, så `hour` saknas ofta i urvalet. Det kan justeras med `max_features`.
- **Varför blir Logistic Regression sämst?** Den klarar bara raka samband. Trafik går upp och ner under dygnet (låg på natten, hög vid rusning), vilket träd hanterar mycket bättre. Körningen gav dessutom en `ConvergenceWarning`, eftersom kolumnerna har olika skala. Med `StandardScaler` i en pipeline kan resultatet bli något bättre.

Svaghet att kontrollera: resultaten bygger på en enda uppdelning. Cross-validation (`cross_val_score`, `cv=5`) visar om Decision Tree vinner konsekvent eller av slump.

7. Nytt target: trängseln om en timme

Gruppen bytte target till `future_congestion_level`, alltså trängselnivån vid sensorns nästa mätning exakt 60 minuter senare. Med det filtret återstår 2 270 av 200 000 rader.

Motivering till bytet (Simons förslag): att förutsäga framtida trängsel är mer användbart än att beskriva läget just nu. Det gör det dessutom tillåtet att använda dagens mätningar (`occupancy_rate`, `avg_speed_kmh` med flera), eftersom de är kända *innan* det vi förutsäger. Läckageproblemet från avsnitt 4 försvinner alltså.

Hur kolumnen skapas

```
df = df.sort_values(["sensor_id", "timestamp"])   # säkerhetsåtgärd, se nedan
df["future_congestion_level"] = df.groupby("sensor_id")
["congestion_level"].shift(-1)
df["next_timestamp"] = df.groupby("sensor_id")["timestamp"].shift(-1)
df["duration_to_next"] = pd.to_datetime(df["next_timestamp"]) -
pd.to_datetime(df["timestamp"])
df = df[df["duration_to_next"].dt.total_seconds() / 60 == 60]
df = df.drop(columns=["next_timestamp", "duration_to_next"])
```

`shift(-1)` hämtar värdet från raden under, separat för varje sensor (`groupby`). Varje rad får alltså "framtidens svar" bredvid sig.

Varför 60-minutersfiltret behövs: tiden mellan två mätningar från samma sensor är lång och oregelbunden. Utan filtret skulle "nästa" kunna betyda allt från samma minut till 11 dagar senare.

Tid till nästa mätning	Värde
Medelvärde	21 h
Median	14,5 h
25:e percentilen	6 h
Minimum	0 h (dubbletter vid samma tidpunkt)
Maximum	11 dagar

160 sensorer har cirka 1 250 mätningar var under tre år, i snitt en mätning per 21 timmar. Det tyder på att datan är syntetisk med slumpmässiga tidpunkter.

Kontroller vi gjort

- **Sortering:** vi körde koden med och utan `sort_values`. Båda gav 2 270 rader, så datan var redan sorterad och den ursprungliga koden gav rätt resultat. Sorteringen behålls som skydd om datan laddas i annan ordning.
- **`drop` måste sparas:** `df.drop(...)` utan `df =` lämnar kolumnerna kvar. `next_timestamp` är text och skulle få modellen att krascha.

8. Klassfördelning för det nya targetet

Klasserna är obalanserade: free_flow utgör cirka 47 % av raderna, ungefär lika mycket som de tre andra klasserna tillsammans.

Trängsel om en timme	Antal rader (cirka)	Andel
free_flow	1 075	47 %
moderate	415	18 %
light	405	18 %
heavy	375	17 %

Siffrorna kommer från Muamers graf, gjord före filtreringen på `data_quality_flag` (totalt 2 270 rader). Med filtret blir det något färre rader, men samma fördelning.

Vad obalansen betyder för analysen

- En modell som alltid gissar free_flow får redan 47 % rätt. Det blir vår första baseline.
- `stratify=y` i `train_test_split` behövs, så att alla klasser finns i samma andel i tränings- och testdatan.
- Accuracy räcker inte ensamt. Vi tittar också på recall per klass i confusion matrix (avsnitt 11).

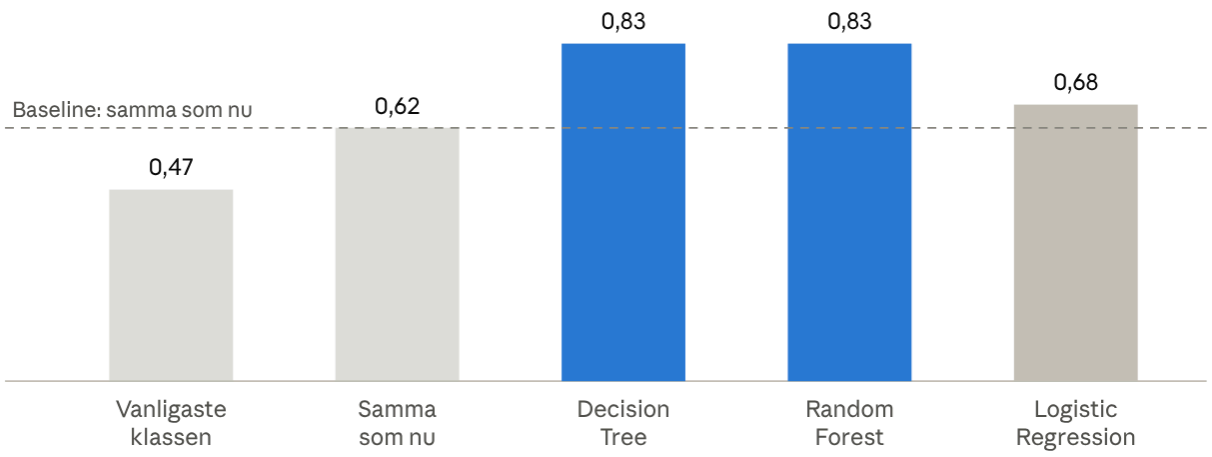
Text eller siffror för klasserna? Muamer visade att medelhastigheten sjunker 13 km/h mellan free_flow och light, men bara 2 km/h mellan light och moderate. Stegen mellan klasserna är alltså inte lika stora. Därför behåller vi klasserna som text och gör klassificering, inte regression. För klassificerarna spelar det ingen roll om klasserna är text eller 0–3, eftersom de ser dem som namn på kategorier.

9. Resultat: modeller mot baselines

Decision Tree och Random Forest förutsäger trängseln om en timme med accuracy 0,83. Det är 21 procentenheter bättre än att gissa att trängseln blir samma som nu (0,62).

Trädmodellerna slår ”samma som nu” med 21 procentenheter

Accuracy på testdatan, trängsel en timme framåt



Från Arezoo_modeller.ipynb · testdata 20 %, random_state=42

Två baselines

- **Vanligaste klassen (0,47):** gissar alltid free_flow. Det är golvet.
- **Samma som nu (0,62):** gissar att trängseln om en timme är densamma som nu. Trafik ändras ofta långsamt, så det är en stark baseline. En modell är bara värd något om den kan förutsäga **förändringar**.

```
dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
samma_som_nu = df.loc[X_test.index, "congestion_level"]
accuracy_score(y_test, samma_som_nu)   # 0.62
```

Indata (X): läget just nu, alltså tid, väder, väg, plats, trafikmätningar och nuvarande trängsel. Mätkolumnerna är tillåtna här, eftersom de mäts en timme *innan* det vi förutsäger.

Tolkning: trädmodellerna kan förutsäga förändringar, till exempel att trafiken ökar inför rusningstid. Logistic Regression (0,68) ligger nära baselinen, eftersom trafikens upp-och-ned-mönster inte är linjärt. Den kördes med `StandardScaler` i en pipeline för att undvika `ConvergenceWarning`.

10. Cross-validation: Random Forest är bäst och stabilast

Med fem omgångar i stället för en enda uppdelning blir Random Forest det säkraste valet: 0,81 ± 0,02, jämfört med 0,79 ± 0,07 för Decision Tree.

Modell	Accuracy (medel av 5)	Spridning (±)	Jämfört med "samma som nu" (0,63)
Random Forest	0,81	0,02	Bättre, stabilt
Decision Tree	0,79	0,07	Bättre, men varierar mer
Logistic Regression	0,55	0,15	Sämre, och instabil

```
for name, m in models.items():
    scores = cross_val_score(m, X, y, cv=5)
    print(name, ":", scores.mean().round(3), "±", scores.std().round(3))
```

Varför cross-validation? Med cirka 2 200 rader blir testdatan bara cirka 450 rader. Ett enda test kan då bli bra eller dåligt av ren tur. `cv=5` delar datan i fem delar och låter varje del vara testdata en gång. Medelvärdet visar hur bra modellen är, och spridningen hur stabil den är.

Tolkning

- Vid en enda uppdelning fick Decision Tree och Random Forest samma accuracy (0,83). Cross-validation visar att Random Forest är lika bra men mycket stabilare. Ett enda träd påverkas mer av vilken data det tränas på (risk för överanpassning), medan skogen jämnar ut felen genom att 100 träd röstar.
- Logistic Regression hamnar under baselinen "samma som nu". En linjär modell klarar inte trafikens mönster.
- Cross-validation ändrade alltså vårt val av modell, från "båda är lika bra" till **Random Forest**.

11. Confusion matrix: var modellen gör fel

Modellen är bäst på ytterklasserna free_flow (91 %) och heavy (86 %). Svårast är moderate (64 %), som ligger mitt emellan och blandas ihop med sina grannklasser.

Andel per verklig klass, Decision Tree på testdatan. Varje rad summerar till 100 %, och diagonalen visar andelen rätt.

Verklig ↓ / Gissad →	free_flow	light	moderate	heavy
free_flow	**91 %**	5 %	0 %	4 %
light	4 %	**76 %**	20 %	0 %

Verklig ↓ / Gissad →	free_flow	light	moderate	heavy
moderate	3 %	21 %	**64 %**	12 %
heavy	4 %	1 %	9 %	**86 %**

```
ConfusionMatrixDisplay.from_estimator(
    best, X_test, y_test,
    labels=["free_flow", "light", "moderate", "heavy"],
    normalize="true", values_format=".0%", cmap="Blues",
)
```

`normalize="true"` visar procent per rad i stället för antal. Då kan klasserna jämföras rättvist trots att free_flow har fler rader.

Tolkning

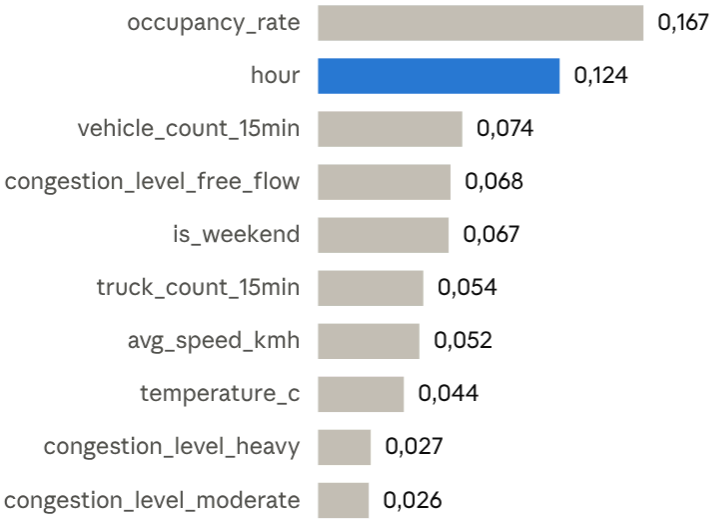
- **Felen hamnar nästan alltid bredvid diagonalen.** Moderate gissas som light (21 %) eller heavy (12 %), och light gissas som moderate (20 %). Modellen gör sällan grova fel, men har svårt att dra gränsen mellan närliggande nivåer. Det stämmer med Muamers observation att hastigheten bara skiljer 2 km/h mellan light och moderate.
- **Några "hopp" över hela skalan:** 4 % free_flow gissas som heavy och tvärtom. De kan bero på plötsliga händelser, som en olycka eller början på rusningstid, som är svåra att se en timme i förväg.

12. Feature importance: vad modellen använder för prognosen

För trängseln om en timme väger trafiken just nu tyngst, med `occupancy_rate` (0,167) överst. Tiden (`hour`, 0,124) är fortfarande viktig, men mindre än i modellen för trängseln just nu (0,381, avsnitt 5).

Trafiken just nu väger tyngst – hour sjunker från 0,38 till 0,12

Feature importance, Random Forest för trängseln om en timme, topp 10 (samma skala som avsnitt 5)



Från Arezoo_modeller.ipynb · Random Forest, feature_importances_, 10 viktigaste kolumnerna

Grupperat

Grupp	Kolumner	Vikt (cirka)
Trafiken just nu	`occupancy_rate`, `vehicle_count_15min`, `truck_count_15min`, `avg_speed_kmh`	0,35
Tid	`hour`, `is_weekend`	0,19
Trängseln just nu	`congestion_level_free_flow`, `_heavy`, `_moderate`	0,12
Väder	`temperature_c`	0,04

Tolkning

- Hur full vägen är just nu säger mest om hur det ser ut om en timme.
- `hour` gör att modellen kan förutse **förändringar**, till exempel att trafiken ökar inför rusningstid. Det förklarar varför modellen slår baselinen "samma som nu".
- Ingen kolumn dominerar: den högsta vikten är 0,167, jämfört med nästan 1,0 när det fanns läckage. Modellen kombinerar flera signaler.
- Nuvarande `congestion_level` delades upp i tre kolumner av `get_dummies`. Var för sig ser de små ut, men tillsammans väger de ungefär lika mycket som `hour`.

13. Korrelation: mätkolumnerna bekräftar läckaget

`occupancy_rate` har korrelationen 0,94 med trängseln just nu, alltså nästan ett perfekt samband. Det bekräftar visuellt varför de fyra mätkolumnerna togs bort i avsnitt 4.

Spearman-korrelation med trängseln just nu (`congestion_level`), på hela den rensade datan (cirka 184 000 rader):

Kolumn	Korrelation	Tolkning
`occupancy_rate`	0,94	Nästan facit
`vehicle_count_15min`	0,80	Starkt
`truck_count_15min`	0,72	Starkt
`is_weekend`	0,54	Mer trängsel på helger
`avg_speed_kmh`	−0,53	Lägre hastighet, mer trängsel
`hour`	0,13	Svagt, se nedan

Väder (`temperature_c` , `precipitation_mm`) och vägtyp (`speed_limit_kmh` , `lane_count`) ligger nära 0.

```
ordning = {"free_flow": 0, "light": 1, "moderate": 2, "heavy": 3}
data = df_full.copy()
data["congestion_level_num"] = data["congestion_level"].map(ordning)
corr = data[kolumner].astype(float).corr(method="spearman")
```

Varför 0–3 här, när klasserna annars är text? Korrelation kräver siffror. Spearman använder bara **ordningen** (heavy är mer än moderate), inte avstånden mellan stegen. Därför gäller inte Muamers invändning om olika stora steg just här. Ändringen görs på en kopia (`data`), så originaldatan påverkas inte.

Tre intressanta iakttagelser

1. **`hour` visar bara 0,13, men var viktigast i modellen (avsnitt 5).** Korrelation mäter bara samband där "mer ger mer". Trafiken går upp och ner under dygnet, och det mönstret missar korrelationen men trädmodellerna hittar. Korrelation visar alltså inte allt.
2. **`is_weekend` är positiv (0,54),** alltså mer trängsel på helger. Det är tvärtom mot verklig trafik och tyder på att datasetet är syntetiskt.
3. **`avg_speed_kmh` och `speed_limit_kmh` har korrelationen 0,78.** Hastigheten beror mycket på vägtypen. Därför blir sambandet mellan hastighet och trängsel bara −0,53. Trängseln är troligen beräknad från hastigheten i förhållande till hastighetsgränsen.

14. Slutsatser

Random Forest kan förutsäga trafikträngseln en timme framåt med accuracy 0,81 ± 0,02 i cross-validation. Det är tydligt bättre än att gissa att trängseln blir samma som nu (0,63).

1. **Ett för bra resultat är en varningssignal.** Den första modellen fick 99,99 %, eftersom `occupancy_rate`, `avg_speed_kmh`, `vehicle_count_15min` och `truck_count_15min` mäts samtidigt som trängseln och i praktiken avslöjar svaret. Vi upptäckte läckaget, testade det steg för steg och tog bort kolumnerna. Korrelationen (0,94 för `occupancy_rate`) bekräftar det.
2. **Utan läckage förutsäger tid, väder och väg trängseln just nu med 0,86–0,90.** Tiden på dygnet (`hour`) är viktigast.
3. **Att förutsäga trängseln om en timme är ett mer användbart problem.** Där får mätningarna vara med, eftersom de är kända i förväg. Trädmodellerna slår baselinen "samma som nu" med cirka 20 procentenheter, vilket betyder att de kan förutse förändringar och inte bara upprepa nuläget.
4. **Random Forest är det bästa valet.** Den är lika träffsäker som Decision Tree på en enda uppdelning, men mycket stabilare i cross-validation (± 0,02 mot ± 0,07). Logistic Regression klarar inte trafikens icke-linjära mönster och hamnar under baselinen.
5. **Modellen gör sällan grova fel.** Felen hamnar mellan grannklasser, särskilt kring moderate (64 % rätt), där gränserna mot light och heavy är otydliga.
6. **Modellen kombinerar trafiken just nu med tiden.** Trafiken just nu väger tyngst, och tiden hjälper modellen att förutse när trängseln ändras.

Viktigaste lärdomen: granska resultat kritiskt. Ett perfekt resultat säger oftare något om datan än om modellen.

15. Begränsningar och nästa steg

Den största begränsningen är datamängden: bara cirka 2 200 av 200 000 rader har en mätning exakt en timme senare.

Begränsningar

- **Lite data för prognosen.** Testdatan blir cirka 450 rader, och resultaten varierar mer. Cross-validation minskar men tar inte bort problemet.
- **Troligen syntetisk data.** Glesa, slumpmässiga tidpunkter (median 14,5 h mellan mätningar) och mer trängsel på helger tyder på det. Resultaten säger därför mer om datasetet än om verklig trafik.
- **Slumpmässig uppdelning i stället för uppdelning efter tid.** För en riktig prognos borde man träna på äldre data och testa på nyare.

- **Obalanserade klasser.** free_flow utgör 47 %, vilket gör accuracy något missvisande. Recall per klass ger en fullare bild.

Möjliga nästa steg

- [ ] Optimera Random Forest med `GridSearchCV` (`n_estimators`, `max_depth`, `max_features`).
- [ ] Testa ett tidsfönster (till exempel 45–90 minuter) i stället för exakt 60 minuter, för att få mer data.
- [ ] Dela upp train/test efter tid och jämför resultatet.
- [ ] Pröva `class_weight="balanced"` för att hjälpa de mindre klasserna, särskilt moderate.

Arbetssätt i gruppen

- Koden ligger i en egen notebook (`Arezoo_modeller.ipynb`) på branchen `arezoo-modeller`, så att den inte krockar med gruppens `traffic.ipynb`.
- Cellerna måste ligga i rätt ordning: en cell kan bara använda variabler som skapats i celler ovanför. Notebooken körs med **Restart Kernel and Run All Cells** innan den pushas.